In [1]:
```
using Revise
include("main.jl")
using .KronMatrixModule
```

In [2]:
```
A = rand(5,4)
B = rand(7,9)
K = KronMatrix2(A, B)
K_16 = KronMatrix2(Float16, A, B)

display("Type of K is $(typeof(K))")
display("Type of K_16 is $(typeof(K_16))")
```

```
"Type of K is KronMatrix2{Float64}"
"Type of K_16 is KronMatrix2{Float16}"
```

In [3]:
```
using LinearAlgebra
K2 = kron(A, B)
display("Float64: $(norm(K2 - full(K)))")

###

# Precision check
display("Float16: $(norm(K2 - full((K_16))))")
display("Type of full K_16 is $(typeof(full(K_16)))")
```

```
"Float64: 0.0"
"Float16: 0.004593760829757954"
"Type of full K_16 is Matrix{Float16}"
```

In [4]:
```
print("Float64 multiplication norm: ")

# multiplication
x = rand(size(K,2),1)
b = K*x
display(norm(b - K2*x))

# multiple columns
x_m = rand(size(K,2),10);
b = K*x_m;
display(norm(b - K2*x_m))

# multiplty on the left
y = rand(10,size(K,1))
z = y*K
display(norm(z - y*K2))

###

print("\n")

# Precision check
print("Float16 norm: ")
x = convert(Matrix{Float16}, x)
b = K_16*x
display(norm(b - K2*x))
display("Type of z is $(typeof(b))")

# multiple columns
x_m = convert(Matrix{Float16}, x_m)
b = K_16*x_m;
display(norm(b - K2*x_m))
display("Type of z is $(typeof(b))")

# multiplty on the left
y = convert(Matrix{Float16}, y)
z = y*K_16
```

```
display(norm(z - y*K2))
display("Type of z is $(typeof(z))")
```

```
Float64 multiplication norm:
4.572178254219406e-15
1.6881232242081456e-14
1.6866622778295818e-14
Float16 norm:
0.010171274985190321
"Type of z is Matrix{Float16}"
0.03649285907478415
"Type of z is Matrix{Float16}"
0.03672951262155779
"Type of z is Transpose{Float16, Matrix{Float16}}"
```

In [5]:

```
# division
A = rand(4,4)
B = rand(7,7)
K = KronMatrix2(A, B)
K2 = kron(A, B)
b = rand(size(K,1),1)
x = K\b
display("Float64: $(norm(x - K2\b)/norm(x))")

###

# Precision check
K_16 = KronMatrix2(Float16, A, B)
b = convert(Matrix{Float16}, b)
x = K_16\b
display("Float16: $(norm(x - K2\b)/norm(x))")
display("Type of x is $(typeof(x))")
```

```
"Float64: 4.822648225599395e-15"
"Float16: 0.005893855681750277"
"Type of x is Vector{Float16}"
```

In [6]:

```
# transpose
display("Float64: $(norm(full(K') - K2'))")

###

# Precision check
display("Float16: $(norm(full(K_16') - K2'))")
```

```
"Float64: 0.0"
"Float16: 0.003287502368964982"
```

In [7]:

```
# svd
U, S, V = svd(K)
print("Float64 types: ")
display(typeof(U))
display(typeof(S))
display(typeof(V))

###

# Precision check
U, S, V = svd(K_16)
print("Float16 types: ")
display(typeof(U))
display(typeof(S))
display(typeof(V))
```

```
Float64 types:
KronMatrix2{Float64}
KronMatrix2{Float64}
KronMatrix2{Float64}
Float16 types:
KronMatrix2{Float16}
```

```
KronMatrix2{Float16}
KronMatrix2{Float16}
```

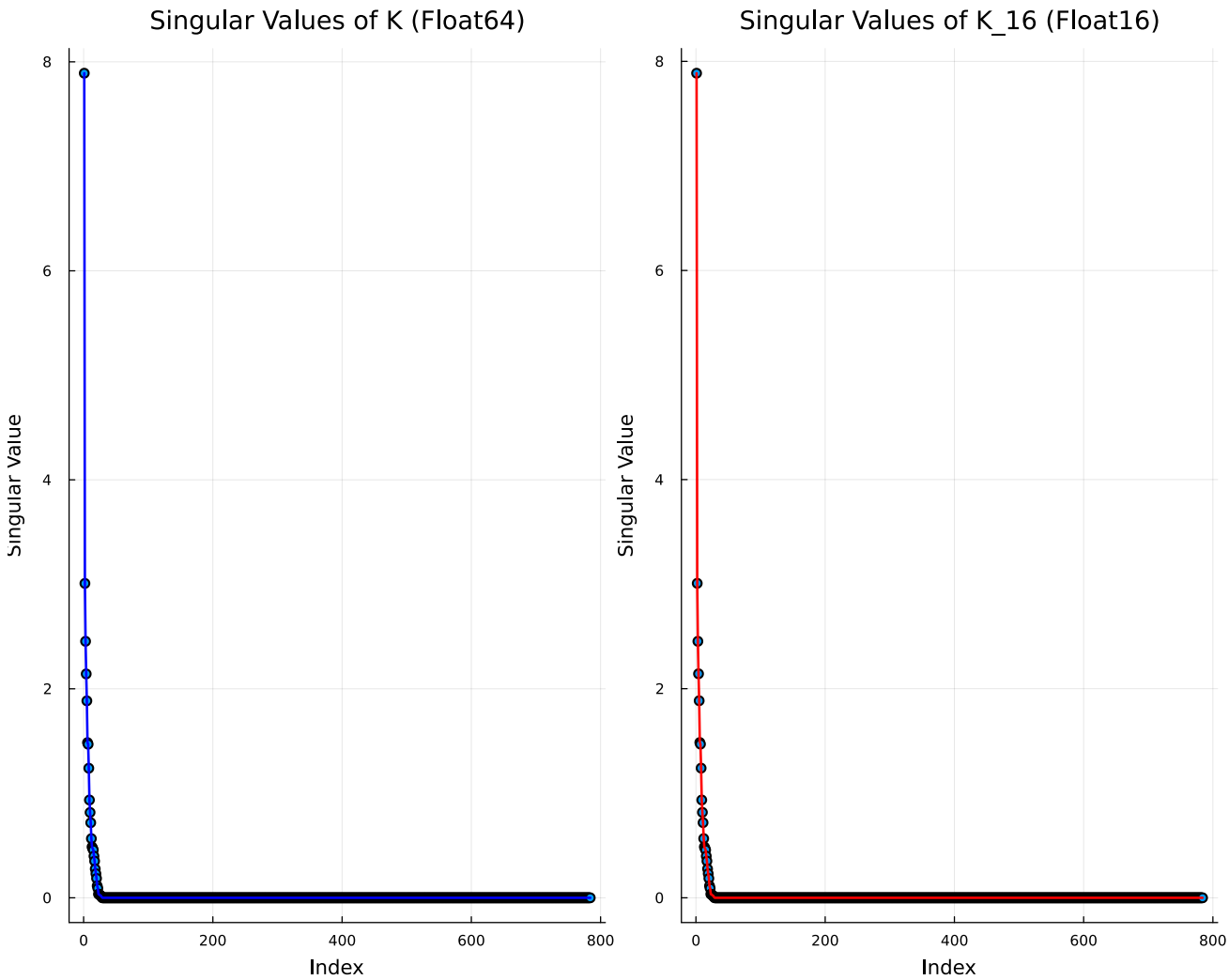
In [27]:

```julia
using Plots

# First SVD
s1 = svd(K; full=false)
p1 = plot(s1;
    seriestype = :scatter,
    line = (:solid, 2, :blue),
    xlabel = "Index",
    ylabel = "Singular Value",
    title = "Singular Values of K (Float64)",
    legend = false,
    fontsize = 18)
plot!(p1, s1, line = (:solid, 2, :blue))  # Optional line overlay

# Second SVD
s2 = svd(K_16; full=false)
p2 = plot(s2;
    seriestype = :scatter,
    line = (:solid, 2, :red),
    xlabel = "Index",
    ylabel = "Singular Value",
    title = "Singular Values of K_16 (Float16)",
    legend = false,
    fontsize = 18)
plot!(p2, s2, line = (:solid, 2, :red))  # Optional line overlay

# Combine
plot(p1, p2, layout = (1, 2), size=(1000, 800))

# the largest singular values of Float16 are slightly bigger than that of Float64.
```

Out[27]: